

SE3082 – Parallel Computing

Exercise 4



IT24103470
Kathisan A

Exercise 4 Details, Code & Output Verification

1. Exercise Task Description

Describe how threads are generated for standard loop parallelization vs when using `#pragma omp parallel for collapse(2)`.

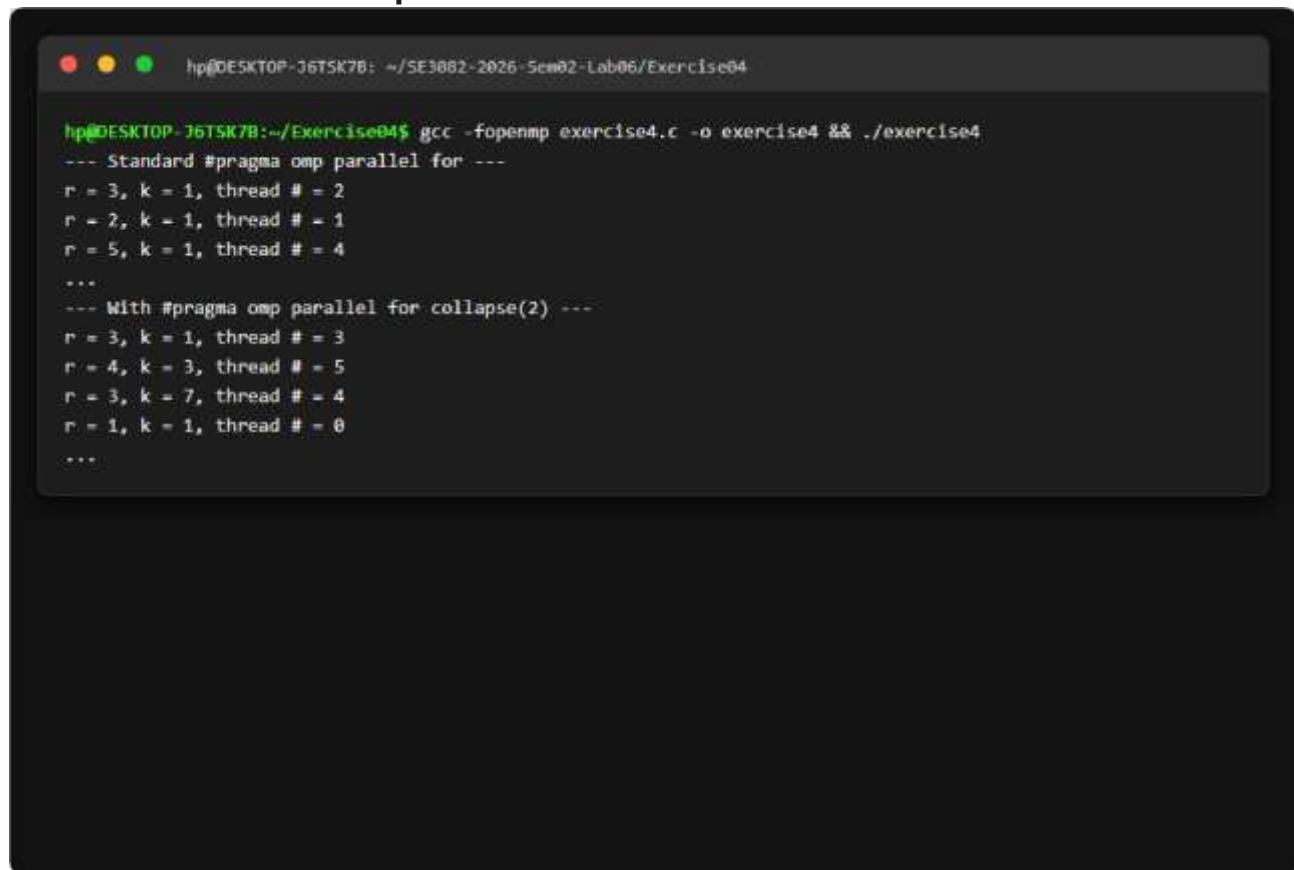
2. Complete C Source Code (exercise4.c)

```
#include <omp.h>
#include <stdio.h>

int main() {
    printf("--- Standard #pragma omp parallel for ---\n");
    #pragma omp parallel for
    for (int r = 1; r <= 5; r++) {
        for (int k = 1; k <= 10; k++) {
            printf("r = %d, k = %d, thread # = %d\n", r, k, omp_get_thread_num());
        }
    }

    printf("\n--- With #pragma omp parallel for collapse(2) ---\n");
    #pragma omp parallel for collapse(2)
    for (int r = 1; r <= 5; r++) {
        for (int k = 1; k <= 10; k++) {
            printf("r = %d, k = %d, thread # = %d\n", r, k, omp_get_thread_num());
        }
    }
    return 0;
}
```

3. Terminal Execution Output Screenshot



```
hp@DESKTOP-J6TSK7B: ~/SE3882-2026-Sem02-Lab06/Exercise04

hp@DESKTOP-J6TSK7B:~/Exercise04$ gcc -fopenmp exercise4.c -o exercise4 && ./exercise4

--- Standard #pragma omp parallel for ---
r = 3, k = 1, thread # = 2
r = 2, k = 1, thread # = 1
r = 5, k = 1, thread # = 4
...
--- With #pragma omp parallel for collapse(2) ---
r = 3, k = 1, thread # = 3
r = 4, k = 3, thread # = 5
r = 3, k = 7, thread # = 4
r = 1, k = 1, thread # = 0
...
```

4. Directive Analysis & Explanation

Explanation of collapse(2):

Standard `#pragma omp parallel for` only parallelizes the outermost `r` loop (5 iterations), while inner `k` loops run sequentially. Adding `collapse(2)` flattens both nested loops into a single virtual loop of 50 iterations ($r \in [1,5], k \in [1,10]$), distributing all 50 iteration tuples across available threads for significantly improved load balancing.